

Static Application Security Testing (SAST) Report

Report generated by: https://github.com/TempeHS/Secure_Architecture_Sandbox_Testing_Environment

Analyses source code for security vulnerabilities without executing the programme

Report Information

Field	Value
Generated	June 17, 2026 at 01:24 AM UTC
Target Path	/workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa
Files Analysed	5
Tools Used	bandit, semgrep

Executive Summary

19 security issues were found and **7 high severity** issues that require immediate attention.

Vulnerability Breakdown

Severity	Count	Percentage
CRITICAL	0	0.0%
HIGH	7	36.8%
MEDIUM	11	57.9%
LOW	1	5.3%
INFO	0	0.0%

Learning Objectives

After reviewing this SAST analysis report, you should be able to:

1. Understand how static code analysis identifies security vulnerabilities
2. Learn to interpret SAST tool outputs and prioritise findings
3. Recognise common code patterns that lead to security issues
4. Apply secure coding practices to prevent vulnerabilities

Detailed Findings

The following 19 security issues were identified:

Finding 1

HIGH flask_debug_true

Tool bandit

Description: A Flask app appears to be run with debug=True, which exposes the Werkzeug debugger and allows the execution of arbitrary code.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/main.py:70

CWE ID: [CWE-94](#)

Educational Explanation

Flask debug mode should not be enabled in production as it can expose sensitive information.

How to Fix This

Set debug=False in production. Use environment variables to control debug mode.

Confidence: MEDIUM

Finding 2

HIGH Data from request is passed to redirect(). This is an open redirect and could be exploited. Consider using 'url_for()' to generate links to known locations. If you must use a URL to unknown pages, consider using 'urlparse()' or similar and checking if the 'netloc' property is the same as your site's host name. See the references for more information.

Tool semgrep

Description: Data from request is passed to redirect(). This is an open redirect and could be exploited. Consider using 'url_for()' to generate links to known locations. If you must use a URL to unknown pages, consider using 'urlparse()' or similar and checking if the 'netloc' property is the same as your site's host name. See the references for more information.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/main.py:16

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 3

HIGH Data from request is passed to `redirect()`. This is an open redirect and could be exploited. Consider using `'url_for()'` to generate links to known locations. If you must use a URL to unknown pages, consider using `'urlparse()'` or similar and checking if the `'netloc'` property is the same as your site's host name. See the references for more information.

Tool `semgrep`

Description: Data from request is passed to `redirect()`. This is an open redirect and could be exploited. Consider using `'url_for()'` to generate links to known locations. If you must use a URL to unknown pages, consider using `'urlparse()'` or similar and checking if the `'netloc'` property is the same as your site's host name. See the references for more information.

Location: `/workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/main.py:31`

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 4

HIGH Data from request is passed to `redirect()`. This is an open redirect and could be exploited. Consider using `'url_for()'` to generate links to known locations. If you must use a URL to unknown pages, consider using `'urlparse()'` or similar and checking if the `'netloc'` property is the same as your site's host name. See the references for more information.

Tool `semgrep`

Description: Data from request is passed to `redirect()`. This is an open redirect and could be exploited. Consider using `'url_for()'` to generate links to known locations. If you must use a URL to unknown pages, consider using `'urlparse()'` or similar and checking if the `'netloc'` property is the same as your site's host name. See the references for more information.

Location: `/workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/main.py:48`

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 5

HIGH **Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option.**

Tool **semgrep**

Description: Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:20

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 6

HIGH **Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option.**

Tool **semgrep**

Description: Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:25

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 7

HIGH **Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option.**

Tool **semgrep**

Description: Avoiding SQL string concatenation: untrusted input concatenated with raw SQL query can result in SQL Injection. In order to execute raw query safely, prepared statement should be used. SQLAlchemy provides TextualSQL to easily used prepared statement with named parameters. For complex SQL composition, use SQL Expression Language or Schema Definition Language. In most cases, SQLAlchemy ORM will be a better option.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:45

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 8

MEDIUM **hardcoded_bind_all_interfaces**

Tool **bandit**

Description: Possible binding to all interfaces.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/main.py:70

CWE ID: [CWE-605](#)

Educational Explanation

Security pattern detected - review for potential vulnerabilities.

How to Fix This

Review the code and implement appropriate security controls.

Confidence: MEDIUM

Finding 9

MEDIUM hardcoded_sql_expressions

Tool bandit

Description: Possible SQL injection vector through string-based query construction.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:20

CWE ID: [CWE-89](#)

Educational Explanation

Imagine a library where you can request books by filling out a form. Normally, you'd write 'Harry Potter' and the librarian would find that book. But what if you wrote 'Harry Potter; also give me all books and customer information'? If the librarian blindly follows your request without questioning the extra commands, you'd get access to everything in the library!

Focus: SQL injection is like tricking a database into running extra commands by disguising them as normal search requests. It's like asking for a book but secretly also asking for the keys to the entire library.

How to Fix This

Replace all dynamic SQL with parameterized queries • Implement input validation for all user inputs • Use ORM frameworks when possible

Confidence: MEDIUM

Finding 10

MEDIUM hardcoded_sql_expressions

Tool bandit

Description: Possible SQL injection vector through string-based query construction.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:25

CWE ID: [CWE-89](#)

Educational Explanation

Imagine a library where you can request books by filling out a form. Normally, you'd write 'Harry Potter' and the librarian would find that book. But what if you wrote 'Harry Potter; also give me all books and customer information'? If the librarian blindly follows your request without questioning the extra commands, you'd get access to everything in the library!

Focus: SQL injection is like tricking a database into running extra commands by disguising them as normal search requests. It's like asking for a book but secretly also asking for the keys to the entire library.

How to Fix This

Replace all dynamic SQL with parameterized queries • Implement input validation for all user inputs • Use ORM frameworks when possible

Confidence: MEDIUM

Finding 11

MEDIUM hardcoded_sql_expressions

Tool bandit

Description: Possible SQL injection vector through string-based query construction.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:45

CWE ID: [CWE-89](#)

Educational Explanation

Imagine a library where you can request books by filling out a form. Normally, you'd write 'Harry Potter' and the librarian would find that book. But what if you wrote 'Harry Potter; also give me all books and customer information'? If the librarian blindly follows your request without questioning the extra commands, you'd get access to everything in the library!

Focus: SQL injection is like tricking a database into running extra commands by disguising them as normal search requests. It's like asking for a book but secretly also asking for the keys to the entire library.

How to Fix This

Replace all dynamic SQL with parameterized queries • Implement input validation for all user inputs • Use ORM frameworks when possible

Confidence: MEDIUM

Finding 12

MEDIUM Running flask app with host 0.0.0.0 could expose the server publicly.

Tool semgrep

Description: Running flask app with host 0.0.0.0 could expose the server publicly.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/main.py:70

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 13

MEDIUM Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug' using system environment variables.

Tool semgrep

Description: Detected Flask app with debug=True. Do not deploy to production with this flag enabled as it will leak sensitive information. Instead, consider using Flask configuration variables or setting 'debug' using system environment variables.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/main.py:70

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 14

MEDIUM Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.

Tool semgrep

Description: Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/templates/signup.html:4

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 15

MEDIUM Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.

Tool semgrep

Description: Manually-created forms in django templates should specify a csrf_token to prevent CSRF attacks.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/templates/success.html:4

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 16

MEDIUM Detected possible formatted SQL query. Use parameterized queries instead.

Tool semgrep

Description: Detected possible formatted SQL query. Use parameterized queries instead.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:20

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 17

MEDIUM Detected possible formatted SQL query. Use parameterized queries instead.

Tool semgrep

Description: Detected possible formatted SQL query. Use parameterized queries instead.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:25

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 18

MEDIUM Detected possible formatted SQL query. Use parameterized queries instead.

Tool **semgrep**

Description: Detected possible formatted SQL query. Use parameterized queries instead.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:45

Educational Explanation

Pattern-based analysis detected a potential security issue. Review the code for proper input validation and security controls.

How to Fix This

Review the code and implement appropriate security controls.

Finding 19

LOW blacklist

Tool **bandit**

Description: Standard pseudo-random generators are not suitable for security/cryptographic purposes.

Location: /workspaces/Secure_Architecture_Sandbox_Testing_Environment/samples/unsecure-pwa/user_management.py:33

CWE ID: [CWE-330](#)

Educational Explanation

Security pattern detected - review for potential vulnerabilities.

How to Fix This

Review the code and implement appropriate security controls.

Confidence: HIGH

Recommendations

General Security Recommendations:

1. Integrate SAST tools into your development pipeline for continuous security scanning
2. Address high and critical severity findings first
3. Review and understand each vulnerability before marking as false positive
4. Use secure coding guidelines to prevent similar issues in the future

• **Priority Actions:**

The following issues require immediate attention:

- **7 High severity issues** - Address within 24-48 hours

Additional Learning Resources

To learn more about security testing and vulnerability management:

- [OWASP Top 10](#) - Most critical web application security risks
- [CWE/SANS Top 25](#) - Most dangerous software weaknesses
- [NIST Cybersecurity Framework](#) - Cybersecurity best practices
- [OWASP Static Analysis Tools](#)

This report was generated by the Secure Architecture Sandbox Testing Environment Security Analysis Platform Report Type: SAST Analysis Generated: June 17, 2026 at 01:24 AM